■■ Sri Lankan Vehicle Appraiser

System Architecture & Tech Stack Reference Guide

Executive Summary

The **Sri Lankan Vehicle Appraiser** is a localized, data-driven valuation and analytics platform. It replaces static guesswork with real-time secondary market intelligence by combining automated web scraping pipelines with dynamic statistical processing. The system evaluates asset worth, adjusts for mileage and age depreciation factor adjustments relative to regional peer data clusters, and renders interactive reports.

1. System Architecture Layers

- **User Interface Layer (Frontend):** Provides a highly interactive, responsive dashboard built using **Streamlit** for inputting vehicle specs, launching dynamic valuation queries, and inspecting statistical reports.
- **Data Scraping & Mining Layer (Ingestion):** Employs **Playwright** for dynamic, browser-rendered listing extractions, alongside a multi-page static crawler using **BeautifulSoup4** & **Requests** (featuring polite 2-second delays for rate-limit protection).
- **Data Cleaning & Engineering Layer:** Uses **Pandas** dataframes combined with python **Regular Expressions (re)** to parse unformatted price tags (e.g. *'Rs. 8,750,000'*) and mileage strings (e.g. *'165,000 km'*) into numeric structures.
- **Analytics & Modeling Layer (Statistical Engine):** Leverages **NumPy** to compute peer-group averages and bounds (20th percentile budget floor and 80th percentile premium ceiling), adjusting value dynamically according to mileage deviation from cluster averages.
- **Persistence & Storage Layer:** Utilizes local structured file storage including **.xlsx** (for broad historical indexes), **.csv** (actively compiled outputs), and **.json** (cached market rates and model-specific annual depreciation ratios).

2. Detailed Technology Component Inventory

Component	Technology / Lib	Role / Usage in Project
Frontend UI	Streamlit	Constructs interactive web app dashboard, tables, controls, and report display cards.
Dynamic Scraping	Playwright	Launches headless Chromium instances to fetch active, client-rendered listing elements.
Static Scraping	BeautifulSoup4 / Requests	Downloads and parses HTML indexes across paginated marketplace sections.
Data Engineering	Pandas	Cleans raw spreadsheets, renames columns, drops NaNs, and handles filtering.

Data Extraction	Regex (re)	Parses price digits and mileage text labels (e.g. 'Rs. 8.75M' or '165,000 km').
Statistical Modeling	NumPy	Calculates mean rates, 20% budget floor, 80% premium ceiling, and user mile-variance bounds.
Data Caching	JSON / CSV	Stores baseline configurations, depreciation matrices, and extracted listings records.